Fortran で学ぶ
オブジェクト指向プログラミング

モジュール・派生型・型束縛手続き・継承・多態性・抽象型

1. なぜオブジェクト指向か
2. モジュールとカプセル化
3. 派生型と型束縛手続き
4. 継承（extends）とオーバーライド
5. 多態性：class・select type・多態的な配列
6. 抽象型と deferred 手続き
7. 実践例：原子核エネルギー計算の部品化
8. 演算子の多重定義と final 手続き
9. コンパイルの方法と落とし穴
10. 早見表

2026年10月　／　対象：Fortran 2003 以降（例は gfortran 13.3, `-std=f2018` で動作確認済み）

1. なぜオブジェクト指向か

FORTRAN 77 までのプログラムは「データ（配列や COMMON ブロック）」と「処理（サブルーチン）」が別々に置かれ、どのサブルーチンがどのデータを書き換えるのかをプログラマが頭で管理する必要がありました。コードが大きくなると、この管理が破綻しやすくなります。

オブジェクト指向プログラミング（OOP）は、**データとそれを扱う処理をひとまとめにした「オブジェクト」**を単位にプログラムを組み立てる考え方です。Fortran では 2003 規格で本格的に導入され、次の 4 つの柱がそろいました。

考え方	意味	Fortran での道具
カプセル化	内部の実装を隠し、決めた入口からだけ操作させる	`module`, `private` / `public`
抽象データ型	データと処理をまとめた「型」を自分で作る	派生型 `type` ＋ 型束縛手続き `procedure`
継承	既存の型を土台に、成分や処理を足した新しい型を作る	`type, extends(親)`
多態性（ポリモーフィズム）	同じ呼び出しで、実際の型に応じた処理が選ばれる	`class(型)`, `select type`, `abstract`, `deferred`

以下では、これらを短い完結したプログラムで一つずつ確認します。どの例も 1 ファイルでそのままコンパイル・実行できます。

2. モジュールとカプセル化

OOP の土台は `module` です。モジュールの中に変数や手続きを置き、`private` 文で既定を「非公開」にしてから、外に見せたいものだけを `public` にします。こうすると、モジュール外のコードは決められた手続き経由でしか内部状態に触れられません。

ex1_module.f90

```
module counter_mod
  implicit none
  private                    ! 既定ですべて非公開にする
  public :: count_up, get_count    ! 外に見せるものだけ公開

  integer :: n = 0           ! モジュール内部の状態（外からは直接触れない）

contains

  subroutine count_up()
    n = n + 1
```

```
  end subroutine count_up

  integer function get_count()
    get_count = n
  end function get_count

end module counter_mod

program ex1
  use counter_mod
  implicit none
  call count_up()
  call count_up()
  print '(a,i0)', 'count = ', get_count()
  ! n = 100   ! <- n は private なのでコンパイルエラーになる
end program ex1
```

実行結果

```
count = 2
```

> **ポイント**　最初に `private` と書く習慣をつけると、公開範囲が「意図して書いたものだけ」になり、後から内部を変更しても利用側が壊れにくくなります。

3. 派生型と型束縛手続き

3.1 派生型（type）

派生型は複数の成分（メンバ）をまとめた自作の型で、C の構造体に相当します。成分には `%` でアクセスします。成分に既定値（`= 0` など）を与えておくと、宣言しただけの変数もその値で初期化されます。

3.2 型束縛手続き（メソッド）

型定義の中に `contains` を書き、その後ろに `procedure :: 名前` を並べると、その手続きが型に結びつきます（他言語の「メソッド」）。呼び出しは `obj%名前(...)` です。

このとき、**呼び出したオブジェクト自身が自動的に第 1 引数として渡されます**（これを passed-object dummy argument と呼びます）。受け取る側は `class(型名)` で宣言します。名前は自由ですが、慣習的に `this` や `self` がよく使われます。

3.3 コンストラクタ

型名を関数のように呼ぶと、成分を順番（または成分名）で指定して値を作れます（構造体コンストラクタ）。入力のチェックや変換をしたいときは、**型と同じ名前の総称インターフェース**を作り、自作の関数を登録します。

ex2_type.f90

```
module nucleus_mod
  implicit none
  private
  public :: nucleus_t

  type :: nucleus_t
     integer :: z = 0      ! 陽子数
     integer :: n = 0      ! 中性子数
   contains
     procedure :: mass_number   ! 型束縛手続き（メソッド）
     procedure :: show
  end type nucleus_t

  ! 型と同じ名前の総称インターフェース = 自作コンストラクタ
  interface nucleus_t
     module procedure new_nucleus
  end interface nucleus_t

contains

  function new_nucleus(z, a) result(self)
    integer, intent(in) :: z, a
    type(nucleus_t) :: self
    if (a < z) error stop 'A < Z は不正'
```

```
    self%z = z
    self%n = a - z
  end function new_nucleus

  integer function mass_number(this)
    class(nucleus_t), intent(in) :: this   ! 渡されるオブジェクト自身
    mass_number = this%z + this%n
  end function mass_number

  subroutine show(this)
    class(nucleus_t), intent(in) :: this
    print '(a,i0,a,i0,a,i0)', 'Z=', this%z, '  N=', this%n, &
                             '  A=', this%mass_number()
  end subroutine show

end module nucleus_mod

program ex2
  use nucleus_mod
  implicit none
  type(nucleus_t) :: u238, pb208

  u238  = nucleus_t(92, 238)          ! 自作コンストラクタ (Z, A)
  pb208 = nucleus_t(z=82, n=126)      ! 構造体コンストラクタ (成分名で指定)

  call u238%show()
  call pb208%show()
  print '(a,i0)', 'A(U-238) = ', u238%mass_number()
end program ex2
```

実行結果

```
Z=92  N=146  A=238
Z=82  N=126  A=208
A(U-238) = 238
```

> **なぜ type ではなく class で受けるのか**　`type(nucleus_t)` だと「ちょうど nucleus_t」しか受け取れません。`class(nucleus_t)` は「nucleus_t と、それを継承したすべての型」を受け取れます。型束縛手続きの第 1 引数は継承に備えて `class` にすることが規格で決まっています。

4. 継承（extends）とオーバーライド

`type, extends(親の型) :: 子の型` と書くと、親の成分と型束縛手続きをすべて引き継いだ新しい型を作れます。子の型では次のことができます。

- 成分や手続きを**追加**する。
- 親と同じ名前の手続きを定義し直して**オーバーライド**する（`procedure :: describe => 子の実装`）。
- 子の中の親部分には `obj%親の型名` でアクセスでき、`call this%親の型名%手続き()` で親の実装を呼べる。

`procedure :: 公開名 => 実装名` の形は「型の外からは `describe` という名前で呼ぶが、中身は `particle_describe` という手続き」という対応づけです。実装名はモジュール内で重複できないので、型ごとに別名をつけておきます。

ex3_extends.f90

```
module particle_mod
  implicit none
  private
  public :: particle_t, charged_t

  type :: particle_t
     character(len=16) :: name = ''
     real(8)           :: mass = 0d0     ! MeV/c^2
   contains
     procedure :: describe => particle_describe
  end type particle_t

  ! particle_t を継承（extends）した子の型
  type, extends(particle_t) :: charged_t
     integer :: charge = 0               ! 電荷（e 単位）
   contains
     procedure :: describe => charged_describe   ! オーバーライド
  end type charged_t

contains

  subroutine particle_describe(this)
    class(particle_t), intent(in) :: this
    print '(a,a,f10.3,a)', trim(this%name), ': m =', this%mass, ' MeV'
  end subroutine particle_describe

  subroutine charged_describe(this)
    class(charged_t), intent(in) :: this
    call this%particle_t%describe()      ! 親の手続きを呼ぶ
    print '(a,i0)', '   charge = ', this%charge
  end subroutine charged_describe

end module particle_mod

program ex3
```

```
  use particle_mod
  implicit none
  type(particle_t) :: n0
  type(charged_t)  :: p

  n0 = particle_t('neutron', 939.565d0)
  p  = charged_t('proton', 938.272d0, 1)  ! 親の成分 → 子の成分の順
  call n0%describe()
  call p%describe()
  print '(a,a)', 'p の親部分の名前: ', trim(p%particle_t%name)
end program ex3
```

実行結果

```
neutron: m =   939.565 MeV
proton: m =   938.272 MeV
   charge = 1
p の親部分の名前: proton
```

> **ポイント**　オーバーライドする手続きの引数の並び・型・intent は親と一致させる必要があります（第 1 引数の `class(...)` だけは子の型に変わります）。

5. 多態性：class・select type・多態的な配列

`class(shape_t)` で宣言した変数は、**宣言時の型（宣言型）**は shape_t でも、**実行時の型（動的型）**として circle_t や rect_t を保持できます。`allocatable` または `pointer` 属性をつけ、次のいずれかで動的型を決めます。

- `allocate(s, source=circle_t(r=2d0))` … 値ごとコピーして確保（最も確実）
- `allocate(circle_t :: s)` … 型だけ指定して確保（成分は既定値）
- `s = rect_t(...)` … 代入で自動確保（Fortran 2008）

こうした変数に対して `s%area()` を呼ぶと、**実行時の動的型に応じた実装が自動的に選ばれます**（動的ディスパッチ）。呼ぶ側は円か長方形かを気にする必要がありません。

逆に、子の型だけが持つ成分（円の半径 `r` など）に触りたいときは `select type` で動的型を判定します。`type is` は型がぴったり一致したとき、`class is` はその型か子孫のとき、`class default` はそれ以外に選ばれます。

ex4_poly.f90

```
module shapes_mod
  implicit none
  private
  public :: shape_t, circle_t, rect_t, shape_box

  type :: shape_t
   contains
     procedure :: area => shape_area
  end type shape_t

  type, extends(shape_t) :: circle_t
     real(8) :: r = 1d0
   contains
     procedure :: area => circle_area
  end type circle_t

  type, extends(shape_t) :: rect_t
     real(8) :: w = 1d0, h = 1d0
   contains
     procedure :: area => rect_area
  end type rect_t

  ! 異なる型を 1 つの配列に入れるための「箱」
  type :: shape_box
     class(shape_t), allocatable :: s
  end type shape_box

contains

  real(8) function shape_area(this)
    class(shape_t), intent(in) :: this
    shape_area = 0d0
  end function shape_area
```

```
  real(8) function circle_area(this)
    class(circle_t), intent(in) :: this
    circle_area = acos(-1d0) * this%r**2
  end function circle_area

  real(8) function rect_area(this)
    class(rect_t), intent(in) :: this
    rect_area = this%w * this%h
  end function rect_area

end module shapes_mod

program ex4
  use shapes_mod
  implicit none
  type(shape_box) :: list(3)
  class(shape_t), allocatable :: s
  integer :: i

  allocate(list(1)%s, source=circle_t(r=2d0))   ! source= の型と値で確保
  allocate(list(2)%s, source=rect_t(w=3d0, h=4d0))
  allocate(circle_t :: list(3)%s)      ! 型を指定して確保（既定値で初期化）

  do i = 1, size(list)
     ! 実行時の動的型に応じて circle_area / rect_area が呼ばれる
     print '(a,i0,a,f8.3)', 'shape ', i, ': area = ', list(i)%s%area()
  end do

  ! select type で動的型を調べ、子の型の成分にアクセスする
  do i = 1, size(list)
     select type (sh => list(i)%s)
     type is (circle_t)
        print '(a,f6.2)', 'circle, r = ', sh%r
     type is (rect_t)
        print '(a,2f6.2)', 'rect,   w,h = ', sh%w, sh%h
     class default
        print '(a)', 'unknown shape'
     end select
  end do

  s = rect_t(w=1d0, h=5d0)             ! 代入による自動確保 (F2008)
  print '(a,l1)', 'same_type_as(s, rect_t()) = ', same_type_as(s, rect_t())
end program ex4
```

実行結果

```
shape 1: area =   12.566
shape 2: area =   12.000
shape 3: area =    3.142
circle, r =   2.00
rect,   w,h =   3.00  4.00
circle, r =   1.00
same_type_as(s, rect_t()) = T
```

> **多態的な配列の注意**　`class(shape_t), allocatable :: a(:)` は作れますが、**配列の全要素が同じ動的型**になります。円と長方形を混ぜたいときは、例のように `class(...), allocatable` 成分を 1 つ持つ「箱」の型（`shape_box`）を作り、その配列を使うのが定石です。

6. 抽象型と deferred 手続き

例 4 の `shape_t` は「面積 0 を返す」というダミーの実装を持っていました。しかし本当は「図形」そのものに面積は定義できず、**「子の型は必ず area を実装すること」という約束だけ**を書きたいはずです。これを表すのが抽象型です。

- `type, abstract :: 型名` … この型の変数は直接作れない（`class` としてのみ使える）。
- `procedure(インターフェース名), deferred :: 手続き名` … 実装を子の型に任せる手続き。
- `abstract interface` … deferred 手続きの引数と戻り値の「形」を定義する。中で親の型などを使うには `import` が必要。

抽象型を継承した子の型は、deferred 手続きを**すべて実装しないとコンパイルエラー**になります（子自身も abstract にする場合を除く）。実装忘れをコンパイラが見つけてくれるのが大きな利点です。

ex5_abstract.f90（抜粋）

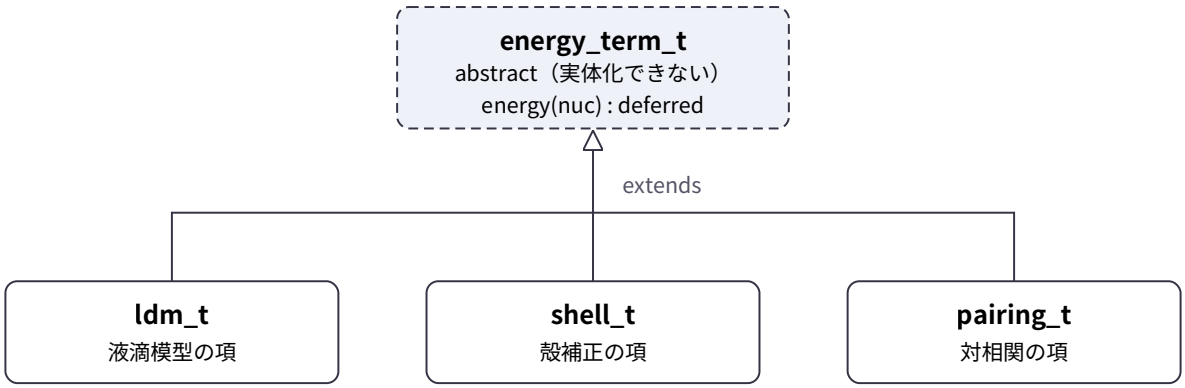
```
! 抽象型: これ自体は実体化できない。「energy を持つ」という約束だけ定める
type, abstract :: energy_term_t
   character(len=24) :: label = ''
 contains
   procedure(energy_iface), deferred :: energy
end type energy_term_t

abstract interface
   function energy_iface(this, nuc) result(e)
     import :: energy_term_t, nucleus_t, dp
     class(energy_term_t), intent(in) :: this
     type(nucleus_t),      intent(in) :: nuc
     real(dp) :: e
   end function energy_iface
end interface
```

次の章では、この抽象型を使って実際に計算を組み立てます。

7. 実践例：原子核エネルギー計算の部品化

原子核のポテンシャルエネルギーを「巨視的（液滴模型）項 ＋ 殻補正 ＋ 対相関補正」の和として計算するプログラムを考えます。各項を、共通の抽象型 `energy_term_t` を継承した型として作ると、全体は次のような構造になります。



calculator_t は class(energy_term_t) の配列を持ち、中身の具体的な型を知らずに energy を合計する

この設計の利点は、**新しい項（例えば別の巨視的模型や BCS による対相関エネルギー）を追加しても、合計を取る `calculator_t` 側は一行も変えなくてよい**ことです。新しい型を作って `calc%add(...)` で登録するだけで済み、模型の差し替えや比較も簡単になります。

> **注意**　以下の数式は OOP の構造を示すための簡略版（トイモデル）です。液滴項は変形を 2 次までの Bohr–Wheeler 型で近似し、殻補正は $\beta_2 \approx 0.25$ 付近にガウス型の谷を置いただけのダミー、対相関は偶奇の経験式です。実際の計算では各型の `energy` の中身を FRLDM・Strutinsky 法・BCS などの本格的な実装に置き換えます。`nucleus_t` には三軸変形用に `gamma` 成分も用意してあります。

ex5_abstract.f90

```
!=============================================================
! 例 5: 抽象型による「エネルギー項」の部品化（原子核の例）
!    ※ 数式は OOP の説明用の簡略版（トイモデル）です
!=============================================================
module energy_term_mod
  implicit none
  private
  public :: dp, nucleus_t, energy_term_t

  integer, parameter :: dp = kind(1d0)

  type :: nucleus_t
     integer  :: z, n
     real(dp) :: beta2 = 0d0, gamma = 0d0   ! 変形パラメータ (gamma は度)
  end type nucleus_t
```

```
  ! 抽象型: これ自体は実体化できない。「energy を持つ」という約束だけ定める
  type, abstract :: energy_term_t
     character(len=24) :: label = ''
   contains
     procedure(energy_iface), deferred :: energy
  end type energy_term_t

  abstract interface
     function energy_iface(this, nuc) result(e)
       import :: energy_term_t, nucleus_t, dp
       class(energy_term_t), intent(in) :: this
       type(nucleus_t),      intent(in) :: nuc
       real(dp) :: e
     end function energy_iface
  end interface

end module energy_term_mod

!-----------------------------------------------------------------
module ldm_mod                          ! 巨視的（液滴）項
  use energy_term_mod
  implicit none
  private
  public :: ldm_t

  type, extends(energy_term_t) :: ldm_t
     real(dp) :: av = 15.75d0, as = 17.8d0, ac = 0.711d0, aa = 23.7d0
   contains
     procedure :: energy => ldm_energy
  end type ldm_t

contains

  function ldm_energy(this, nuc) result(e)
    class(ldm_t),    intent(in) :: this
    type(nucleus_t), intent(in) :: nuc
    real(dp) :: e, a, bs, bc
    a  = real(nuc%z + nuc%n, dp)
    bs = 1d0 + 0.4d0 * nuc%beta2**2      ! 表面エネルギーの変形依存（2次まで）
    bc = 1d0 - 0.2d0 * nuc%beta2**2      ! クーロンエネルギーの変形依存
    e  = - this%av * a + this%as * a**(2d0/3d0) * bs                  &
         + this%ac * nuc%z * (nuc%z - 1) / a**(1d0/3d0) * bc          &
         + this%aa * (nuc%n - nuc%z)**2 / a
  end function ldm_energy

end module ldm_mod

!-----------------------------------------------------------------
module shell_mod                        ! 殻補正項（ここではダミー）
  use energy_term_mod
  implicit none
  private
  public :: shell_t

  type, extends(energy_term_t) :: shell_t
     real(dp) :: amp = -8d0, width = 0.12d0, beta0 = 0.25d0
   contains
     procedure :: energy => shell_energy
  end type shell_t
```

```
contains

  function shell_energy(this, nuc) result(e)
    class(shell_t),  intent(in) :: this
    type(nucleus_t), intent(in) :: nuc
    real(dp) :: e
    ! 本来は Strutinsky 法で計算する。ここでは変形 beta0 付近に谷を置くだけ
    e = this%amp * exp(-((nuc%beta2 - this%beta0) / this%width)**2)
  end function shell_energy

end module shell_mod

!---------------------------------------------------------------
module pairing_mod                    ! 対相関項（偶奇項の簡略版）
  use energy_term_mod
  implicit none
  private
  public :: pairing_t

  type, extends(energy_term_t) :: pairing_t
     real(dp) :: ap = 11.18d0
   contains
     procedure :: energy => pairing_energy
  end type pairing_t

contains

  function pairing_energy(this, nuc) result(e)
    class(pairing_t), intent(in) :: this
    type(nucleus_t),  intent(in) :: nuc
    real(dp) :: e, delta
    delta = this%ap / sqrt(real(nuc%z + nuc%n, dp))
    if (mod(nuc%z, 2) == 0 .and. mod(nuc%n, 2) == 0) then
       e = -delta
    else if (mod(nuc%z, 2) == 1 .and. mod(nuc%n, 2) == 1) then
       e = +delta
    else
       e = 0d0
    end if
  end function pairing_energy

end module pairing_mod

!---------------------------------------------------------------
module calculator_mod                 ! 項を束ねて合計する
  use energy_term_mod
  implicit none
  private
  public :: calculator_t

  type :: term_box
     class(energy_term_t), allocatable :: t
  end type term_box

  type :: calculator_t
     type(term_box), allocatable :: terms(:)
   contains
     procedure :: add   => calc_add
     procedure :: total => calc_total
```

```
    procedure :: report
  end type calculator_t

contains

  subroutine calc_add(this, term, label)
    class(calculator_t),  intent(inout) :: this
    class(energy_term_t), intent(in)    :: term
    character(*),         intent(in)    :: label
    type(term_box) :: box
    allocate(box%t, source=term)       ! 動的型ごとコピーして保持
    box%t%label = label
    if (.not. allocated(this%terms)) allocate(this%terms(0))
    this%terms = [this%terms, box]     ! 配列の末尾に追加
  end subroutine calc_add

  function calc_total(this, nuc) result(e)
    class(calculator_t), intent(in) :: this
    type(nucleus_t),     intent(in) :: nuc
    real(dp) :: e
    integer :: i
    e = 0d0
    do i = 1, size(this%terms)
       e = e + this%terms(i)%t%energy(nuc)  ! どの項かを気にせず呼べる
    end do
  end function calc_total

  subroutine report(this, nuc)
    class(calculator_t), intent(in) :: this
    type(nucleus_t),     intent(in) :: nuc
    integer :: i
    do i = 1, size(this%terms)
       print '(2x,a24,f12.3,a)', this%terms(i)%t%label, &
             this%terms(i)%t%energy(nuc), ' MeV'
    end do
    print '(2x,a24,f12.3,a)', 'total', this%total(nuc), ' MeV'
  end subroutine report

end module calculator_mod

!-----------------------------------------------------------------
program ex5
  use energy_term_mod
  use ldm_mod
  use shell_mod
  use pairing_mod
  use calculator_mod
  implicit none
  type(calculator_t) :: calc
  type(nucleus_t)    :: nuc
  integer :: i

  call calc%add(ldm_t(),     'macroscopic (LDM)')
  call calc%add(shell_t(),   'shell correction')
  call calc%add(pairing_t(), 'pairing')

  nuc = nucleus_t(z=68, n=98, beta2=0.25d0)
  print '(a)', 'Er-166, beta2 = 0.25'
  call calc%report(nuc)
```

```
  print '(/,a)', ' beta2      E_total [MeV]'
  do i = 0, 10
     nuc%beta2 = 0.05d0 * i
     print '(f6.2, f16.3)', nuc%beta2, calc%total(nuc)
  end do
end program ex5
```

実行結果

```
Er-166, beta2 = 0.25
  macroscopic (LDM)          -1352.882 MeV
  shell correction              -8.000 MeV
  pairing                       -0.868 MeV
                     total    -1361.750 MeV

 beta2      E_total [MeV]
  0.00        -1359.928
  0.05        -1360.078
  0.10        -1360.529
  0.15        -1361.632
  0.20        -1362.661
  0.25        -1361.750
  0.30        -1357.803
  0.35        -1351.914
  0.40        -1345.952
  0.45        -1340.643
  0.50        -1335.634
```

設計の読みどころ

- **モジュールを項ごとに分割**：`ldm_mod`・`shell_mod`・`pairing_mod` は互いに依存せず、`energy_term_mod` にだけ依存します。実際の開発ではそれぞれ別ファイルにします（9 章）。
- **パラメータは型の成分として持つ**：`ldm_t(av=15.5d0)` のように係数を変えたオブジェクトを複数作れ、パラメータセットの比較が容易になります。グローバル変数を使わないので、並列化（OpenMP など）とも相性が良くなります。
- **`allocate(box%t, source=term)`**：引数で受け取った多態的なオブジェクトを、動的型ごとコピーして保持しています。
- **`this%terms(i)%t%energy(nuc)`**：中身が液滴項か殻補正かを知らずに呼べる、多態性の核心部分です。

8. 演算子の多重定義と final 手続き

型束縛の `generic` を使うと、自作の型に対して `+` や `*` などの演算子、あるいは `.dot.` のような独自演算子を定義できます。ベクトルや行列、複素数的な量を扱うときにコードが数式に近くなります。

- `generic :: operator(+) => add_vv` … `a + b` と書くと `add_vv(a, b)` が呼ばれる。
- スカラー × ベクトル（`2d0 * a`）のように**自分が第 2 引数**になる場合は `pass(v)` で「どの引数が自分か」を指定する。
- 同じ名前の演算子に複数の実装を登録でき、引数の型で自動的に選ばれる。代入 `assignment(=)` も同様に多重定義できる。

`final :: 手続き名` は、オブジェクトが消えるとき（スコープを抜ける・`deallocate` されるなど）に自動で呼ばれる後始末用の手続きです。ポインタで確保したメモリの解放やファイルのクローズに使います。`allocatable` 成分は自動で解放されるので、多くの場合 final は不要です。

ex6_operator.f90

```
module vec_mod
  implicit none
  private
  public :: vec3_t, operator(.dot.)

  type :: vec3_t
     real(8) :: x(3) = 0d0
   contains
     procedure, private :: add_vv
     procedure, private :: mul_sv
     procedure, pass(v), private :: mul_vs
     generic :: operator(+) => add_vv          ! 演算子の多重定義
     generic :: operator(*) => mul_sv, mul_vs
     procedure :: norm
     final :: vec3_final                       ! 後始末（デストラクタ）
  end type vec3_t

  interface operator(.dot.)                     ! 独自の演算子
     module procedure dot_vv
  end interface

contains

  function add_vv(a, b) result(c)
    class(vec3_t), intent(in) :: a, b
    type(vec3_t) :: c
    c%x = a%x + b%x
  end function add_vv

  function mul_sv(v, s) result(c)       ! v * s
    class(vec3_t), intent(in) :: v
    real(8),       intent(in) :: s
    type(vec3_t) :: c
    c%x = s * v%x
```

```
  end function mul_sv

  function mul_vs(s, v) result(c)       ! s * v  (pass(v) で第2引数が自分)
    real(8),       intent(in) :: s
    class(vec3_t), intent(in) :: v
    type(vec3_t) :: c
    c%x = s * v%x
  end function mul_vs

  real(8) function dot_vv(a, b)
    type(vec3_t), intent(in) :: a, b
    dot_vv = sum(a%x * b%x)
  end function dot_vv

  real(8) function norm(this)
    class(vec3_t), intent(in) :: this
    norm = sqrt(sum(this%x**2))
  end function norm

  subroutine vec3_final(this)
    type(vec3_t), intent(inout) :: this
    ! ここでファイルを閉じる・ポインタを解放する等を行う
  end subroutine vec3_final

end module vec_mod

program ex6
  use vec_mod
  implicit none
  type(vec3_t) :: a, b, c
  a = vec3_t([1d0, 2d0, 2d0])
  b = vec3_t([0d0, 1d0, 0d0])
  c = 2d0 * a + b * 3d0
  print '(a,3f6.1)', 'c     = ', c%x
  print '(a,f6.1)',  '|a|   = ', a%norm()
  print '(a,f6.1)',  'a.b   = ', a .dot. b
end program ex6
```

実行結果

```
c     =    2.0   7.0   4.0
|a|   =    3.0
a.b   =    2.0
```

9. コンパイルの方法と落とし穴

9.1 複数ファイルのコンパイル順

モジュールをコンパイルすると `.mod` ファイル（インターフェース情報）が作られ、それを `use` するファイルはその後にコンパイルする必要があります。7 章の例をファイルに分けた場合、依存の根元から順に並べます。

コンパイル例（bash）

```
# 依存される側から順にコンパイル（-c はオブジェクトファイルだけ作る）
gfortran -c energy_term_mod.f90
gfortran -c ldm_mod.f90 shell_mod.f90 pairing_mod.f90
gfortran -c calculator_mod.f90
gfortran -c main.f90
gfortran -o nuclear energy_term_mod.o ldm_mod.o shell_mod.o \
         pairing_mod.o calculator_mod.o main.o

# 開発中はチェックを強めておくと安心
gfortran -std=f2018 -Wall -fcheck=all -g -fbacktrace ...
```

ファイルが増えてきたら Makefile、CMake（Fortran のモジュール依存を自動解決できる）、または Fortran Package Manager（`fpm`）を使うと順番を気にせずに済みます。

9.2 よくある落とし穴

症状・誤り	原因と対処
型束縛手続きで「passed-object は class で宣言せよ」というエラー	第 1 引数を `type(...)` にしている。`class(...)` に直す。
子の型の成分に `obj%r` でアクセスできない	変数の宣言型が親（`class(shape_t)`）なので、コンパイラは子の成分を知らない。`select type` で取り出す。
異なる型を 1 つの配列に入れられない	多態的配列は全要素が同じ動的型。「箱」の派生型の配列を使う（5 章）。
abstract interface 内で型が見つからない	インターフェース本体は外側のスコープが見えない。`import :: 型名, dp` を書く。
多態的な成分への代入で値がおかしい	コンパイラの古い版では `box%s = circle_t(...)` のような代入が正しく動かないことがある。`allocate(box%s, source=...)` を使うのが確実（実際、本書の確認に使った gfortran 13.3 でもこの形で不具合が出たため、例 4 は source= に統一した）。
実装名の衝突	型束縛名（`area`）は型ごとに同じでよいが、実装の手続き名（`circle_area` など）はモジュール内で一意にする。
多態性のせいで遅くならないか	動的ディスパッチの費用は 1 回の呼び出しごとにわずか。最内側ループ（格子点ごとの演算など）では型束縛呼び出しを避け、配列をまとめて渡す粒度で設計すれば問題になりにくい。

9.3 設計の指針

- まずは「モジュール ＋ 派生型 ＋ 型束縛手続き」だけで十分役に立つ。継承・抽象型は**差し替え可能な部品が複数ある場面**（模型の比較、ポテンシャルの種類、ソルバーの選択など）で使う。
- グローバルな `COMMON` やモジュール変数で状態を共有する代わりに、状態をオブジェクトの成分として持たせると、テストや並列化がしやすくなる。
- 継承の階層は浅く保つ（2〜3 段まで）。深い継承より、オブジェクトを成分として持つ「合成」（7 章の `calculator_t` が項を持つ形）の方が見通しが良いことが多い。

10. 早見表

やりたいこと	書き方
非公開を既定にする	`private` ／ `public :: 名前`
型を定義する	`type :: t` … `end type t`
メソッドを付ける	`contains` の後に `procedure :: m`／`procedure :: m => 実装`
自分自身を受け取る	`class(t), intent(in) :: this`
自分が第 1 引数でない	`procedure, pass(v) :: m`／渡さない場合は `nopass`
コンストラクタ	`t(成分...)`／`interface t; module procedure new_t; end interface`
継承する	`type, extends(t) :: u`
親の実装を呼ぶ	`call this%t%m()`
多態的な変数	`class(t), allocatable :: x`（または `pointer`）
動的型で確保	`allocate(x, source=u(...))`／`allocate(u :: x)`
動的型で分岐	`select type (y => x)` ／ `type is (u)` ／ `class is (t)` ／ `class default`
型の比較	`same_type_as(a, b)`／`extends_type_of(a, b)`
実装を子に強制する	`type, abstract :: t` ＋ `procedure(iface), deferred :: m`
演算子を定義する	`generic :: operator(+) => add`／`assignment(=)`
後始末	`final :: cleanup`（引数は `type(t)`）
子でのオーバーライドを禁止	`procedure, non_overridable :: m`

> **次の一歩**　本書の例 5 をファイル分割し、`shell_t` の `energy` を本物の殻補正計算に置き換えるところから始めると、自分のコードに OOP を取り入れる感覚がつかめます。大規模になってきたら、モジュールの実装部分を別ファイルに分ける `submodule`（Fortran 2008）も調べてみてください。再コンパイルの範囲を減らせます。